

Codage du texte

Comment le texte est-il représenté dans un ordinateur ?

A	é	愛	😎
41	C3 A9	E6 84 9B	F0 9F 98 8E

Nicholas Wolff · CC BY-SA 4.0

Sommaire

1. **Codages du texte en nombres**
   ASCII, Character Sets, Unicode, UTF-8
2. **Coder et décoder en UTF-8**
   Du caractère aux octets, et retour
3. **Un problème (presque) résolu**
   Les anciens codages ont la vie dure

Un codage par signal électrique, antérieur à l'informatique

Activité · 6 min

Connaissez-vous déjà des manières de coder du texte pour le transmettre à travers un câble électrique ?

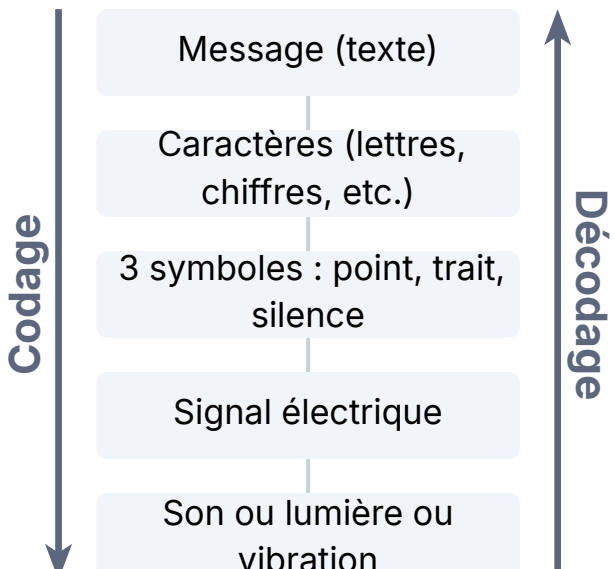
Écouter

Exemple : le morse

Code morse international

1. Un point dure une unité.
2. Un trait dure trois unités.
3. Entre les éléments d'une même lettre : une unité.
4. Entre deux lettres : trois unités.
5. Entre deux mots : sept unités.

A	• —	N	— •	0	— — — — —
B	— • • •	O	— — —	1	• — — — —
C	— • — •	P	• — — •	2	• • — — —
D	— • •	Q	— — • —	3	• • • — —
E	•	R	• — •	4	• • • • —
F	• • — •	S	• • •	5	• • • • •
G	— — •	T	—	6	— • • • •
H	• • • •	U	• • —	7	— — • • •
I	• •	V	• • • —	8	— — — • •
J	• — — —	W	• — —	9	— — — — •
K	— • —	X	— • • —		
L	• — • •	Y	— • — —		
M	— —	Z	— — • •		



Le texte dans un ordinateur et entre ordinateurs

- Un texte est une séquence de caractères. Un caractère : une lettre, un chiffre, un signe de ponctuation, l'espace, un retour à la ligne…
- Dans un ordinateur, tout est représenté en binaire (avec des 0 et des 1).
- Nous savons déjà comment représenter les nombres en binaire.
- Pour représenter un texte, il suffit de trouver une **convention** pour faire correspondre chaque caractère à un nombre.
- Le texte devient alors une séquence de nombres, un par caractère, qu'on lit l'un après l'autre.

Nous allons explorer plusieurs de ces conventions.

Activité · 2 min

Avez-vous une idée de convention pour faire correspondre chaque caractère à un nombre ?

ASCII

American **S**tandard **C**ode for **I**nformation **I**nterchange – Standard créé en 1963

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0_	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	TAB	LF	VT	FF	CR	SO	SI
1_	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2_	␣	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3_	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4_	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5_	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7_	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

Caractères de contrôle, ne s'affichent pas.

␣ : l'espace.

Pour **coder** un caractère : sa ligne donne le premier symbole hexadécimal, sa colonne le deuxième.

Par exemple K : ligne **4_**, colonne **B**, K se code **4B**.

Pour **décoder** un nombre hexadécimal : le premier symbole donne la ligne, le deuxième la colonne.

Par exemple **61** : ligne **6_**, colonne **1**, c'est le caractère a.

ASCII : exercices

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0_	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	TAB	LF	VT	FF	CR	SO	SI
1_	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2_	␣	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3_	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4_	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5_	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7_	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

Binaire	Hexa
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

Activité · 8 min

1. Coder les lettres de votre prénom en hexadécimal à l'aide de la table.
2. Décoder ce texte binaire (convertir vers l'hexadécimal en utilisant la table de droite, puis de l'hexadécimal au texte en utilisant la table ASCII) : `01010011 01001111 01010011`
3. Combien de bits sont utilisés par caractère ?
4. Peut-on coder la phrase « Un âne est-il passé par là ? » à l'aide de cette table (justifier la réponse) ?

ASCII : exercices

1. Coder les lettres de votre prénom en hexadécimal à l'aide de la table.

On utilise la table ASCII pour trouver le nombre hexadécimal correspondant à chaque lettre. Par exemple, « Alan » devient la séquence de 4 octets : `41 6C 61 6E`

2. Décoder ce texte binaire.

- En ASCII, chaque caractère occupe un octet : on découpe le texte tous les 8 bits.
- On commence par convertir chaque octet en hexadécimal.
- Puis on cherche chaque caractère dans la table ASCII.

Binaire	`0101'0011`	`0100'1111`	`0101'0011`
Hexadécimal	`5 3`	`4 F`	`5 3`
Caractères	**S**	**O**	**S**
Texte		**SOS**	

3. Combien de bits sont utilisés par caractère ?

Le dernier caractère de la table ASCII, en bas à droite, a le plus grand nombre : $(7F)_{16}$, soit $(0111'1111)_2$. Même pour lui, le bit de gauche vaut 0 : ce bit n'est jamais utilisé. Il reste $(111'1111)_2$, sur 7 bits.

Donc : en ASCII, **7 bits** sont utilisés par caractère.

4. Peut-on coder la phrase « Un âne est-il passé par là ? » à l'aide de cette table (justifier la réponse) ?

Non, car on n'a aucun moyen de coder les caractères â, é et à. Le standard ASCII est conçu pour l'anglais.

Character Sets

- ASCII est conçu pour l'anglais : impossible d'y coder â, é ou à, et encore moins le grec ou le thaï.
- À partir de 1987 sont créés des **Character Sets** : Latin 1, puis le grec, le cyrillique, l'arabe, l'hébreu, le thaï, etc.
- ASCII n'utilise que **7 bits**, soit 128 caractères. Un octet en a **8** : 256 nombres, donc 128 de plus.
- Un Character Set **étend** ASCII : il garde la table ASCII pour les nombres de $(00)_{16}$ à $(7F)_{16}$, et y ajoute, de $(80)_{16}$ à $(FF)_{16}$, les caractères d'une langue ou d'un système d'écriture.

$(00)_{16}$ à $(7F)_{16}$	$(80)_{16}$ à $(FF)_{16}$
ASCII	grec : α β Σ Ω
identique dans tous les Character Sets	thaï : ก ข ค
	etc.

Character Set : Latin 1

Le Character Set Latin 1 :

- De $(00)_{16}$ à $(7F)_{16}$: la table ASCII, inchangée.
- De $(80)_{16}$ à $(9F)_{16}$: d'autres caractères de contrôle, qui ne s'affichent pas.
- De $(A0)_{16}$ à $(FF)_{16}$: les caractères ajoutés, des lettres accentuées et d'autres symboles.

Activité · 2 min

Quelles langues peut-on écrire avec ce codage ?

Et quelles langues ne peut-on pas écrire ?

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0_																
1_																
2_	␣	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3_	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4_	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5_	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7_	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	
8_																
9_																
A_	NBSP	¡	¢	£	¤	¥	¦	§	¨	©	ª	«	¬	SHY	®	¯
B_	°	±	²	³	´	µ	¶	·	¸	¹	º	»	¼	½	¾	¿
C_	À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
D_	Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
E_	à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
F_	ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ÿ

En gris : caractères de contrôle, ne s'affichent pas.

Défauts des Character Sets

Character Set grec

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0_																
1_																
2_	␣	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3_	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4_	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5_	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7_	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	
8_																
9_																
A_	NBSP	‘	’	£	€	₯	¦	§	¨	©	ͺ	«	¬	SHY		—
B_	°	±	²	³	΄	΅	Ά	·	Έ	Ή	Ί	»	Ό	½	Ύ	Ώ
C_	ΐ	A	B	Γ	Δ	E	Z	H	Θ	I	K	Λ	M	N	Ξ	O
D_	Π	P		Σ	T	Y	Φ	X	Ψ	Ω	Ϊ	Ϋ	ά	έ	ή	ί
E_	ΰ	α	β	γ	δ	ε	ζ	η	θ	ι	κ	λ	μ	ν	ξ	ο
F_	π	ρ	ς	σ	τ	υ	φ	χ	ψ	ω	ϊ	ϋ	ό	ύ	ώ	

Character Set thaï

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0_																
1_																
2_	␣	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3_	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4_	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5_	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7_	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	
8_																
9_																
A_	NBSP	ก	ข	ฃ	ค	ฅ	ฆ	ง	จ	ฉ	ช	ซ	ฌ	ญ	ฎ	ฏ
B_	ฐ	ฑ	ฒ	ณ	ด	ต	ถ	ท	ธ	น	บ	ป	ผ	ฝ	พ	ฟ
C_	ภ	ม	ย	ร	ฤ	ล	ฦ	ว	ศ	ษ	ส	ห	ฬ	อ	ฮ	ฯ
D_	ะ	ั	า	ำ	ิ	ี	ึ	ื	ุ	ู	ฺ					฿
E_	เ	แ	โ	ใ	ไ	ๅ	ๆ	็	่	้	๊	๋	์	ํ	๎	๏
F_	๐	๑	๒	๓	๔	๕	๖	๗	๘	๙	๚	๛				

Activité · 3 min

1. Comment sera compris le caractère grec Σ si on l’interprète dans le Character Set thaï ?
2. Peut-on coder avec un même Character Set le texte « Socrate s’écrit Σωκράτης en grec » ?
3. Peut-on faire un Character Set pour le chinois ?

Défauts des Character Sets

Character Set grec

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0_																
1_																
2_	␣	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3_	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4_	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5_	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7_	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	
8_																
9_																
A_	NBSP	‘	’	£	€	₯	¦	§	¨	©	¸	«	¬	SHY		—
B_	°	±	²	³	΄	΅	Ά	·	Έ	Ή	Ί	»	Ό	½	Ύ	Ώ
C_	ΐ	A	B	Γ	Δ	E	Z	H	Θ	I	K	Λ	M	N	Ξ	O
D_	Π	P		Σ	T	Y	Φ	X	Ψ	Ω	Ϊ	Ϋ	ά	έ	ή	ί
E_	ΰ	α	β	γ	δ	ε	ζ	η	θ	ι	κ	λ	μ	ν	ξ	ο
F_	π	ρ	ς	σ	τ	υ	φ	χ	ψ	ω	ϊ	ϋ	ό	ύ	ώ	

Character Set thaï

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0_																
1_																
2_	␣	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3_	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4_	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5_	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7_	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	
8_																
9_																
A_	NBSP	ก	ข	ฃ	ค	ฅ	ฆ	ง	จ	ฉ	ช	ซ	ฌ	ญ	ฎ	ฏ
B_	ฐ	ฑ	ฒ	ณ	ด	ต	ถ	ท	ธ	น	บ	ป	ผ	ฝ	พ	ฟ
C_	ภ	ม	ย	ร	ฤ	ล	ฦ	ว	ศ	ษ	ส	ห	ฬ	อ	ฮ	ฯ
D_	ะ	ั	า	ำ	ิ	ี	ึ	ื	ุ	ู	ฺ					฿
E_	เ	แ	โ	ใ	ไ	ๅ	ๆ	็	่	้	๊	๋	์	ํ	๎	๏
F_	๐	๑	๒	๓	๔	๕	๖	๗	๘	๙	๚	๛				

- Il faut se mettre d’accord en avance sur comment interpréter les nombres qu’on va échanger : le nombre $(D3)_{16}$ est Σ en grec, mais ◌ำ en thaï.
- Impossible de mélanger dans un même document des langues de Character Sets différents, comme le français et le grec.
- 8 bits ne suffisent pas à représenter de nombreux systèmes d’écriture : le chinois, le japonais, le coréen ont chacun leur codage à plusieurs octets, incompatibles entre eux.

Unicode

Unicode est un *standard* informatique qui permet des échanges de textes dans différentes langues, à un niveau mondial. Il vise au codage de texte écrit en donnant à tout caractère de *n'importe quel système d'écriture* un nom et un *identifiant numérique*, et ce de manière unifiée, *quels que soient la plateforme informatique ou le logiciel utilisé*.

Cet identifiant, le **nombre Unicode** du caractère, va de $\texttt{(0)}_{16}$ à $\texttt{(10FFFF)}_{16}$.

Le projet démarre en 1987 ; la première version est publiée en 1991.

Unicode aujourd'hui

Dans la 18ᵉ version du standard (qui date de 2026), on répertorie 172'808 caractères différents.

A é Σ ω ع א 7 €

愛 が 한 پ ?   ᚻ  ᛋ

Nouveautés de la version 18 : blog.emojipedia.org/whats-new-in-unicode-18-0

Unicode : attribuer des nombres aux caractères

Le standard Unicode se contente d'attribuer un nombre à chaque caractère.

Par exemple, la phrase :

â n e ‰ ␣ 🐶

… se convertit en la séquence de nombres suivante :

$(E2)_{16}$ $(6E)_{16}$ $(65)_{16}$ $(2030)_{16}$ $(20)_{16}$ $(1F436)_{16}$

Mais un ordinateur ne stocke et n'échange que des **octets** : il faudra, à un moment, représenter ces nombres sous forme d'octets.

Unicode : représentation sous forme d'octets

â	n	e	‰	␣	🐶
$(E2)_{16}$	$(6E)_{16}$	$(65)_{16}$	$(2030)_{16}$	$(20)_{16}$	$(1F436)_{16}$
1110'0010	110'1110	110'0101	10'0000'0011'0000	10'0000	1'1111'0100'0011'0110

Le nombre de **bits** nécessaires pour représenter chacun de ces caractères est :

8	7	7	14	6	17

Activité · 3 min

- Combien d'**octets** faut-il pour représenter l'espace ? le n ? le â ? le ‰ ? le 🐶 ?
- Comment représenter cette séquence de nombres en **octets** ? Avez-vous des propositions, des idées ?

Une proposition (trop) simple

Proposition : écrire chaque nombre sur le moins d'octets possible, puis mettre les octets bout à bout.

â	n	e	‰	␣	🐶
$(E2)_{16}$	$(6E)_{16}$	$(65)_{16}$	$(2030)_{16}$	$(20)_{16}$	$(1F436)_{16}$
E2	6E	65	20 30	20	01 F4 36

La phrase devient la séquence d'octets : `E2 6E 65 20 30 20 01 F4 36`

Activité · 3 min

En recevant seulement cette séquence d'octets, peut-on retrouver la phrase ?

Une proposition (trop) simple

Proposition : écrire chaque nombre sur le moins d'octets possible, puis mettre les octets bout à bout.

â	n	e	‰	␣	🐶
$(E2)_{16}$	$(6E)_{16}$	$(65)_{16}$	$(2030)_{16}$	$(20)_{16}$	$(1F436)_{16}$
E2	6E	65	20 30	20	01 F4 36

La phrase devient la séquence d'octets : **E2 6E 65 20 30 20 01 F4 36**

Non : rien dans la séquence n'indique où un caractère s'arrête. Les octets du ‰, par exemple, peuvent se lire de deux façons :

20 30 → $(2030)_{16}$ → ‰

20 | 30 → $(20)_{16}$ $(30)_{16}$ → ␣ 0

Il faut une façon d'écrire les nombres qui dise où chaque caractère commence et s'arrête.

Un format de transformation : UTF-8

Il faut donc une règle pour écrire chaque nombre Unicode en octets, qui permette de relire la séquence sans ambiguïté : un *format de transformation*.

UTF-8 (1992)
Unicode **T**ransformation **F**ormat – **8** bits

- Comme dans notre proposition, un caractère prend un nombre variable d'octets, ici de **un à quatre** ; mais chaque octet indique sa place dans le caractère, et la séquence se relit sans ambiguïté.
- D'autres formats existent : par exemple UTF-32 qui écrit chaque caractère sur 4 octets (facile à relire, mais 4 fois plus long qu'en UTF-8 pour un texte simple)

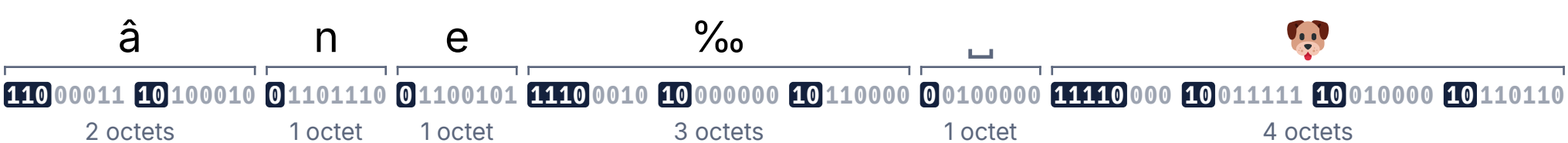
UTF-8 : un codage à taille variable

Selon la taille de son nombre Unicode, un caractère prend en UTF-8 l'une de ces quatre formes. Les *marqueurs*, sur fond foncé, sont fixés par UTF-8 ; les cases grises reçoivent les bits du nombre Unicode.

Nombre d'octets	Octet 1	Octet 2	Octet 3	Octet 4
1	**0**xxxxxxx			
2	**110**xxxxx	**10**xxxxxx		
3	**1110**xxxx	**10**xxxxxx	**10**xxxxxx	
4	**11110**xxx	**10**xxxxxx	**10**xxxxxx	**10**xxxxxx

- Le premier octet annonce la longueur du caractère : il commence par **0** pour un caractère d'un seul octet, et sinon par autant de **1** que le caractère a d'octets, suivis d'un **0** : **110**, **1110** ou **11110**.
- Les octets suivants commencent tous par **10**.

Notre phrase d'exemple s'écrit en 12 octets, qui se relisent sans ambiguïté :



UTF-8 : les bits utilisables

Les cases grises d'un caractère sont ses *bits utilisables* : ceux qui reçoivent son nombre Unicode. Plus un caractère a d'octets, plus il a de bits utilisables, et plus son nombre Unicode peut être grand.

Nombre d'octets	Octet 1	Octet 2	Octet 3	Octet 4
1	`0` ▢▢▢▢▢▢▢			
2	`110` ▢▢▢▢▢	`10` ▢▢▢▢▢▢		
3	`1110` ▢▢▢▢	`10` ▢▢▢▢▢▢	`10` ▢▢▢▢▢▢	
4	`11110` ▢▢▢	`10` ▢▢▢▢▢▢	`10` ▢▢▢▢▢▢	`10` ▢▢▢▢▢▢

Activité · 3 min

Combien de caractères de 2 octets différents peut-on représenter ?

Commencer par compter les bits utilisables d'un caractère de 2 octets. Donner votre réponse sous forme d'une puissance de 2.

UTF-8 : les bits utilisables

Les cases grises d'un caractère sont ses *bits utilisables* : ceux qui reçoivent son nombre Unicode. Plus un caractère a d'octets, plus il a de bits utilisables, et plus son nombre Unicode peut être grand.

Nombre d'octets	Octet 1	Octet 2	Octet 3	Octet 4
1	`0`□□□□□□□			
2	`110`□□□□□	`10`□□□□□□		
3	`1110`□□□□	`10`□□□□□□	`10`□□□□□□	
4	`11110`□□□	`10`□□□□□□	`10`□□□□□□	`10`□□□□□□

Un caractère de 2 octets a 5 bits utilisables dans le premier octet et 6 dans le second : **11 bits**.
Avec 11 bits, on peut écrire 2^{11} = 2'048 **nombres différents**.

Sommaire

1 **Codages du texte en nombres**
ASCII, Character Sets, Unicode, UTF-8

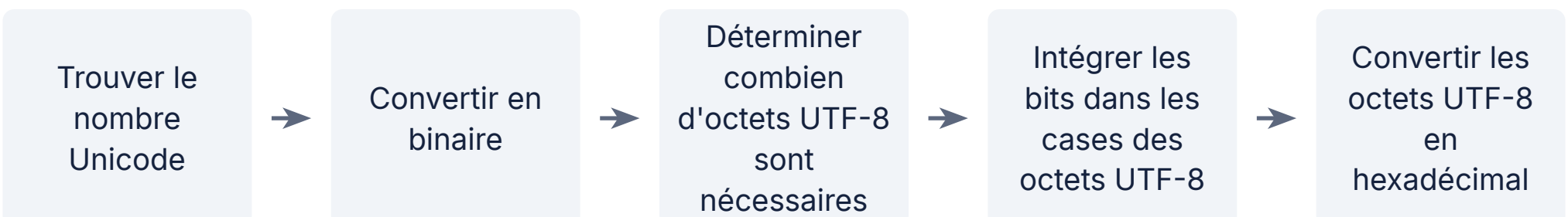
2 **Coder et décoder en UTF-8**
Du caractère aux octets, et retour

3 **Un problème (presque) résolu**
Les anciens codages ont la vie dure

Coder la lettre Ω en UTF-8 : les phases

On cherche à coder le caractère **Ω** en UTF-8.

Il faut procéder en 5 phases :



Trouver le nombre Unicode

Trouver le nombre Unicode → Convertir en binaire → Déterminer combien d'octets UTF-8 sont nécessaires → Intégrer les bits dans les cases des octets UTF-8 → Convertir les octets UTF-8 en hexadécimal

On cherche à coder le caractère **Ω** en UTF-8.

On utilise ri.nwolff.info/unicode-converter pour trouver le nombre Unicode de **Ω** :

Nombre Unicode de **Ω** : $(3A9)_{16}$

Convertir en binaire

Trouver le nombre Unicode → **Convertir en binaire** → Déterminer combien d'octets UTF-8 sont nécessaires → Intégrer les bits dans les cases des octets UTF-8 → Convertir les octets UTF-8 en hexadécimal

Le nombre Unicode du caractère **Ω** est $(3A9)_{16}$.

On convertit en binaire :

```
  3     A     9
0011  1010  1001
```

Le nombre Unicode de **Ω** s'écrit donc `0011'1010'1001` en binaire.

Hexa	Binaire
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

Déterminer combien d'octets UTF-8 sont nécessaires

Trouver le nombre Unicode → Convertir en binaire → **Déterminer combien d'octets UTF-8 sont nécessaires** → Intégrer les bits dans les cases des octets UTF-8 → Convertir les octets UTF-8 en hexadécimal

Le nombre Unicode de **Ω** s'écrit `0011'1010'1001` en binaire.
Sans les 0 à gauche, qui ne comptent pas, `11'1010'1001` : il contient **10 bits** d'information.

On cherche dans la table ci-dessous le plus petit nombre d'octets UTF-8 dont les bits utilisables suffisent pour ces 10 bits.

Nombre d'octets	Octet 1	Octet 2	Octet 3	Octet 4	Nombre de bits utilisables
1	`0`xxxxxxx				7
2	`110`xxxxx	`10`xxxxxx			11
3	`1110`xxxx	`10`xxxxxx	`10`xxxxxx		16
4	`11110`xxx	`10`xxxxxx	`10`xxxxxx	`10`xxxxxx	21

- Sur 1 octet UTF-8, on ne peut faire entrer que 7 bits, parce que le premier bit est forcé à `0` : ça ne convient pas.
- Sur 2 octets UTF-8, on peut faire entrer 11 bits (5 dans le premier octet, 6 dans le deuxième), ce qui est *suffisant* pour représenter 10 bits. On va donc coder **Ω** sur 2 octets.

Intégrer les bits dans les cases des octets UTF-8

Trouver le nombre Unicode → Convertir en binaire → Déterminer combien d'octets UTF-8 sont nécessaires → **Intégrer les bits dans les cases des octets UTF-8** → Convertir les octets UTF-8 en hexadécimal

Le nombre Unicode de **Ω** s'écrit **1110101001** en binaire : il contient 10 bits d'information.

Ces 10 bits entrent dans cette forme de 2 octets UTF-8 : **110**`_____` **10**`_____`

On écrit les bits du nombre sous les cases, en partant de la droite : `1110` `101001`

Chaque bit monte dans sa case ; les cases vides reçoivent des `0` : **110**`01110` **10**`101001`

Résultat : **1110101001** se code sur ces deux octets UTF-8 : **110**`01110` **10**`101001`.

Convertir les octets UTF-8 en hexadécimal

Trouver le nombre Unicode → Convertir en binaire → Déterminer combien d'octets UTF-8 sont nécessaires → Intégrer les bits dans les cases des octets UTF-8 → **Convertir les octets UTF-8 en hexadécimal**

On convertit les octets UTF-8 en hexadécimal :

```
1100 1110    1010 1001
  C    E       A    9
```

Résultat : **Ω** se code `CE A9` en UTF-8.

Binaire	Hexa
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

Coder la lettre Ω en UTF-8 : résumé

1. Le nombre Unicode du caractère **Ω** est $(3A9)_{16}$.
2. $(3A9)_{16}$ s'écrit `1110101001` en binaire.
3. Ce nombre binaire fait 10 bits. 10 bits entrent dans 2 octets UTF-8 : `110_____ 10______`
4. On intègre `1110101001` dans ces 2 octets : `11001110 10101001`
5. `11001110 10101001` s'écrit `CE A9` en hexadécimal.

Le caractère Ω se code `CE A9` en UTF-8.

UTF-8 : exercice de codage

Le nombre Unicode du caractère € est $(20AC)_{16}$.

Activité · 8 min

1. Combien de bits d'information faut-il pour écrire ce nombre Unicode ? (Il faut convertir depuis l'hexadécimal en utilisant la table à droite et compter les bits.)
2. Combien d'octets doit-on utiliser en UTF-8 pour coder ce nombre binaire ? Utiliser le tableau ci-dessous :

Nombre d'octets	Octet 1	Octet 2	Octet 3	Octet 4	Nombre de bits utilisables
1	0xxxxxxx				7
2	110xxxxx	10xxxxxx			11
3	1110xxxx	10xxxxxx	10xxxxxx		16
4	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx	21

3. Construire les octets UTF-8 de € en binaire, à nouveau à l'aide du tableau ci-dessus.
4. Finalement, convertir ces octets en hexadécimal à l'aide de la table à droite.

Binaire	Hexa
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

UTF-8 : exercice de codage

1. Combien de bits d'information faut-il pour écrire ce nombre Unicode ?

On convertit $(20AC)_{16}$ en binaire à l'aide de la table :

2	0	A	C
0010	0000	1010	1100

Sans les 0 à gauche, `10'0000'1010'1100` : ce nombre binaire est fait de **14 bits**.

2. Combien d'octets doit-on utiliser en UTF-8 ?

Ces 14 bits sont codables sur **3 octets** :

Nombre d'octets	Octet 1	Octet 2	Octet 3	Octet 4	Nombre de bits utilisables
1	0				7
2	110	10			11
3	1110	10	10		16
4	11110	10	10	10	21

3. Construire les octets UTF-8 de € en binaire.

On écrit les bits sous les cases, en partant de la droite.

La forme :	1110	10	10
Les bits :		10	000010	101100
Avec des 0 :	1110 0010	10 000010	10 101100

4. Convertir ces octets en hexadécimal.

1110 0010	1000 0010	1010 1100
E 2	8 2	A C

€ se code `E2 82 AC` en UTF-8.

UTF-8 : compter les caractères

Nombre d'octets	Octet 1	Octet 2	Octet 3	Octet 4
1	0			
2	110	10		
3	1110	10	10	
4	11110	10	10	10

Activité · 3 min

Vous recevez un texte complet de 10 octets de long codé en ASCII. Combien y a-t-il de caractères dans ce texte ?

```
42 72 6F 75 69 6C 6C 61 72 64
```

Vous recevez un texte complet de 10 octets de long codé en UTF-8. Combien y a-t-il de caractères dans ce texte ?

```
C3 A9 74 C3 A9 20 F0 9F 98 8E
```

UTF-8 : compter les caractères

Nombre d'octets	Octet 1	Octet 2	Octet 3	Octet 4
1	0			
2	110	10		
3	1110	10	10	
4	11110	10	10	10

Activité

Vous recevez un texte complet de 10 octets de long codé en ASCII. Combien y a-t-il de caractères dans ce texte ?

10 caractères.

42	72	6F	75	69	6C	6C	61	72	64
B	r	o	u	i	l	l	a	r	d

Vous recevez un texte complet de 10 octets de long codé en UTF-8. Combien y a-t-il de caractères dans ce texte ?

5 caractères.

C3 A9	74	C3 A9	20	F0 9F 98 8E
é	t	é	␣	😎

Un défaut d'UTF-8 (par rapport à ASCII, par exemple) est qu'il faut parcourir tout le texte pour savoir le nombre de caractères qu'il contient : il ne suffit pas de compter le nombre d'octets reçus.

Compter les caractères sans tout décoder

Les 4 premiers bits d'un octet, c'est-à-dire son **premier symbole hexadécimal**, suffisent pour reconnaître son marqueur, et donc pour savoir où commence chaque caractère :

Premier symbole	Marqueur	L'octet est…
0 à 7	`0`	un caractère d'un octet
8 à B	`10`	la suite d'un caractère
C, D	`110`	le début d'un caractère de 2 octets
E	`1110`	le début d'un caractère de 3 octets
F	`11110`	le début d'un caractère de 4 octets

Par exemple, la séquence de la diapo précédente :

`C3 A9 74 C3 A9 20 F0 9F 98 8E`

5 débuts de caractère : 5 caractères, sans passer par le binaire.

Pour savoir *quel* caractère c'est, il faut ensuite passer par le binaire : les diapos suivantes.

Décoder un caractère

On veut décoder la séquence `C3 A9 74 C3 A9 20 F0 9F 98 8E`. Le premier symbole des octets (voir « Compter les caractères sans tout décoder ») découpe la séquence en caractères ; on décode le premier, `C3 A9`.

On écrit ses 2 octets en binaire : `110`00011 `10`101001

On enlève les marqueurs et on rassemble les bits du nombre Unicode : 00011101001

Sans les 0 à gauche, par groupes de 4 bits, puis en hexadécimal : $(1110'1001)_2 = (E9)_{16}$

Sur ri.nwolff.info/unicode-converter, ce nombre Unicode est le caractère : é

C’est le chemin inverse du codage.

Décoder toute la séquence

On décode chaque caractère de la même façon. Un caractère d'un octet se lit directement dans la table ASCII.

Octets UTF-8	En binaire	Nombre Unicode	Caractère
C3 A9	110**00011** 10**101001**	$(E9)_{16}$	é
74	0**1110100**	$(74)_{16}$	t
C3 A9	110**00011** 10**101001**	$(E9)_{16}$	é
20	0**0100000**	$(20)_{16}$	␣
F0 9F 98 8E	11110**000** 10**011111** 10**011000** 10**001110**	$(1F60E)_{16}$	😎

La séquence de 10 octets représente le texte de 5 caractères : « **été** 😎 ».

UTF-8 : exercice de décodage

Activité · 8 min

Vous recevez le texte suivant codé en UTF-8 :

```
50 69 C3 B1 61 74 61
```

1. À l'aide du premier symbole hexadécimal de chaque octet, déterminer combien de caractères sont écrits dans ce texte.
2. En utilisant la table ASCII, décoder les caractères qui sont codés sur un octet.
3. Pour chaque caractère manquant : écrire ses octets en binaire, enlever les marqueurs (tableau des formes UTF-8), rassembler les bits du nombre Unicode et les convertir en hexadécimal, puis trouver le caractère sur ri.nwolff.info/unicode-converter.
4. Quel mot avez-vous reçu ?

Tableau des formes UTF-8

Nombre d'octets	Octet 1	Octet 2	Octet 3	Octet 4
1	0xxxxxxx			
2	110xxxxx	10xxxxxx		
3	1110xxxx	10xxxxxx	10xxxxxx	
4	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx

Table ASCII

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0_	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	TAB	LF	VT	FF	CR	SO	SI
1_	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2_	␣	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3_	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4_	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5_	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7_	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

UTF-8 : exercice de décodage

1. Combien de caractères sont écrits dans ce texte ?

Le premier symbole de chaque octet suffit (voir « Compter les caractères sans tout décoder ») :

`50 69 C3 B1 61 74 61`

Il y a donc **6 caractères** : 5 caractères d'un octet chacun, et `C3 B1`, qui forment un seul caractère (voir 3.).

2. Décoder les caractères qui sont codés sur un octet.

À l'aide de la table ASCII, on arrive à décoder partiellement le message :

P i `C3 B1` a t a

3. Décoder les caractères manquants.

`C` annonce un caractère de 2 octets ; en binaire :

`110`00011 `10`110001

Les bits du nombre Unicode, sans les 0 à gauche : $(1111'0001)_2$ = $(F1)_{16}$.

Sur ri.nwolff.info/unicode-converter, c'est le caractère « **ñ** ».

4. Quel mot avez-vous reçu ?

50	69	C3 B1	61	74	61
P	i	ñ	a	t	a

À noter : le message a une longueur de 7 octets, mais ne contient que 6 caractères. C'est normal : UTF-8 est un codage à taille variable.

Retour à la page de titre

Vous pouvez maintenant lire la ligne de la page de titre :



Chaque caractère, quel que soit son système d'écriture, a un nombre Unicode, et UTF-8 écrit ce nombre en 1 à 4 octets.

Sommaire

1. **Codages du texte en nombres**
   ASCII, Character Sets, Unicode, UTF-8
2. **Coder et décoder en UTF-8**
   Du caractère aux octets, et retour
3. **Un problème (presque) résolu**
   Les anciens codages ont la vie dure

De l’ASCII à UTF-8

ASCII
1963
128 caractères : l’anglais.
Ni accents, ni autres systèmes d’écriture.

↓

Character Sets
1987
256 caractères, une table par système d’écriture ou groupe de langues.
Une table à la fois, et il faut savoir laquelle.

↓

Unicode
1991
Un nombre pour chaque caractère de tous les systèmes d’écriture.
Des nombres, pas encore des octets.

↓

UTF-8
1992
Ces nombres en octets, dans une séquence qui se relit sans ambiguïté.

UTF-8 aujourd’hui

99 %

des sites Web utilisent UTF-8

(statistique d’octobre 2026)

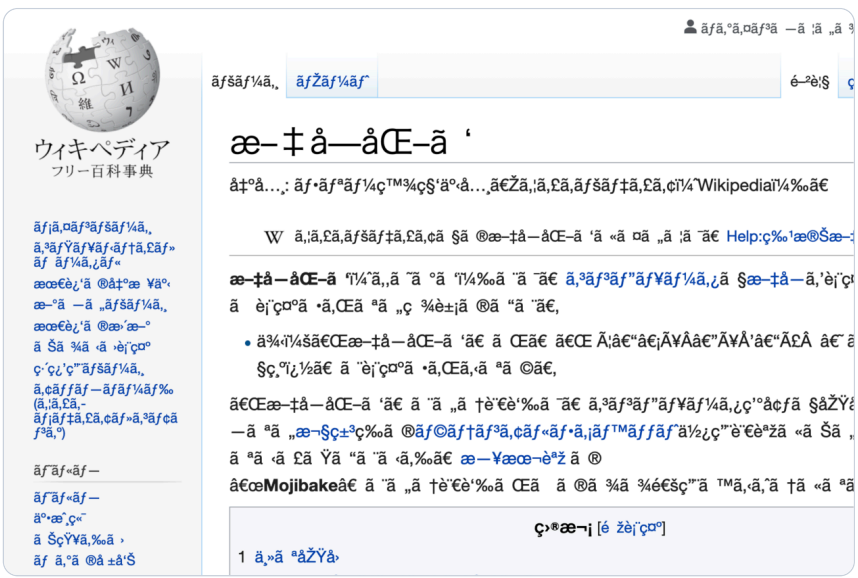
Le problème du texte est donc résolu ? Presque : il reste des endroits où l’on n’écrit encore qu’en ASCII, et des malentendus.

L’ASCII est encore partout

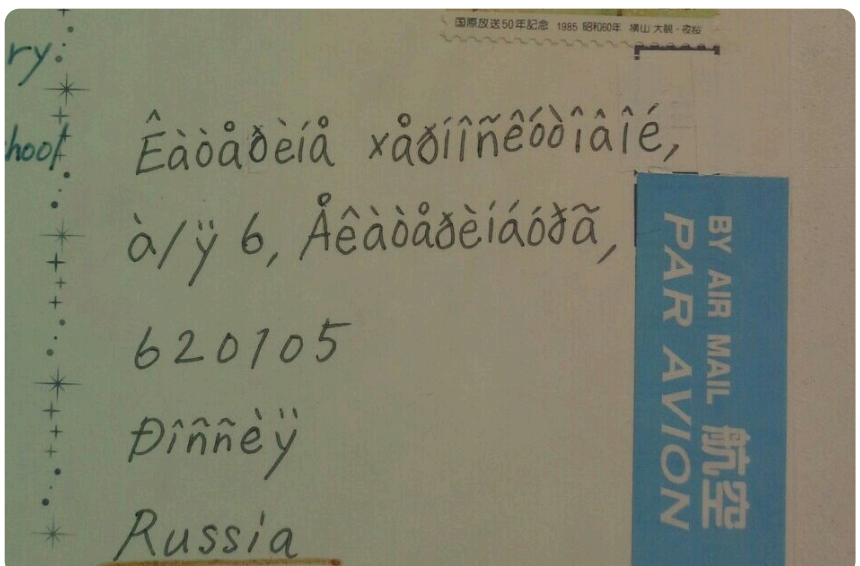
- **Billets d’avion** : les systèmes de réservation des compagnies aériennes n’admettent que les lettres A à Z. Un billet ne peut pas porter le nom MÜLLER.
- **Passeports** : la zone lisible par machine du passeport n’admet que les lettres A à Z, les chiffres et le signe <. Le passeport porte donc deux noms : MÜLLER en clair, MUELLER dans cette zone. C’est ce second nom que le billet doit reprendre.
- **Adresses e-mail** : les adresses avec accents, comme `rené@exemple.ch`, existent depuis 2012, mais peu de services les acceptent : on écrit `rene@exemple.ch`.
- **Adresses Web** : les noms de domaine avec accents existent, mais ils sont rares et convertis en ASCII : l’Académie française est sur `academie-francaise.fr`, où l’on lirait « frankaise ». Dans le reste de l’adresse, un caractère non ASCII s’écrit avec ses octets UTF-8, chacun précédé de % : `fr.wikipedia.org/wiki/Français` devient `…/wiki/Fran%C3%A7ais`.

Mojibake

Le texte illisible qu'on obtient en lisant des octets avec la mauvaise convention a un nom : en japonais *mojibake* (文字化け, « caractères transformés »), en russe *krakozyabry* (кракозябры).



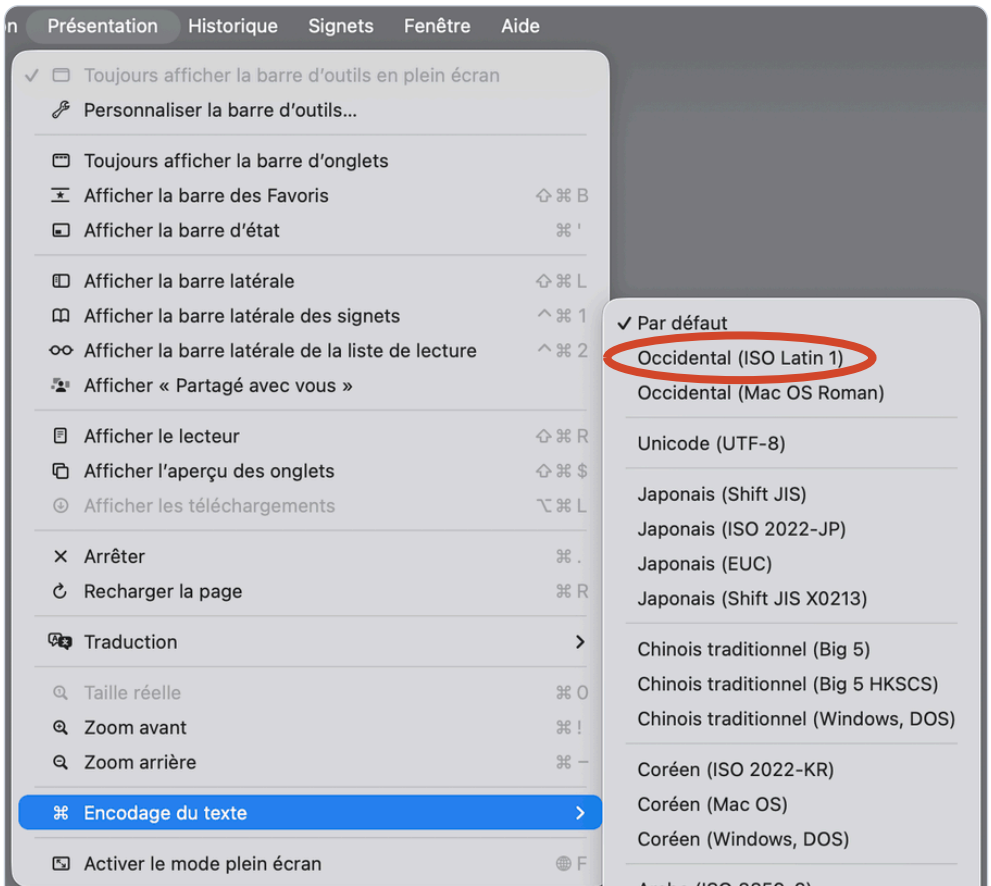
Wikipédia en japonais, écrit en UTF-8, lu comme du Latin 1.



Une carte pour la Russie : l'adresse, écrite dans le Character Set russe, s'affichait au Japon comme du Latin 1, et a été recopiée à la main telle quelle. Elle est arrivée.

Démonstration dans Safari

1. Dans **Safari**, ouvrir la page fr.wikipedia.org/wiki/Gymnase_de_Beaulieu.
2. Choisir un autre codage, par exemple « Occidental (ISO Latin 1) » (menu Présentation → Encodage du texte). Que voyez-vous ?



Sources

Contenu et outils

Wikipédia – Code Morse international
fr.wikipedia.org

Wikipédia – Caractère (typographie)
fr.wikipedia.org

Wikipédia – American Standard Code for Information Interchange
fr.wikipedia.org

ISO – la norme ISO 8859-1:1987, Latin 1
iso.org

Wikipédia – ISO/CEI 8859
fr.wikipedia.org

Convertisseur Unicode
ri.nwolff.info

ridiculousfish – Hex Fiend
hexfiend.com

Unicode et UTF-8

Wikipédia – Unicode
fr.wikipedia.org

Unicode Consortium – l'histoire d'Unicode
unicode.org

Unicode Consortium – Unicode 1.0
unicode.org

Unicode Consortium – Unicode 18.0
unicode.org

Compart – le caractère U+00E9, é
compart.com

Wikipédia – UTF-8
fr.wikipedia.org

W3Techs – l'usage d'UTF-8 sur le Web
w3techs.com

L'ASCII encore là, les malentendus

OACI – Doc 9303, documents de voyage lisibles à la machine
icao.int

IETF – RFC 6530, les adresses e-mail internationalisées (2012)
rfc-editor.org

Wikipédia – Mojibake
fr.wikipedia.org

Microsoft – bulletin MS00-078, la faille d'IIS (2000)
learn.microsoft.com

Images

Nationaal Archief – les caractères d'une machine à écrire japonaise, 1938 (sans restriction connue)
commons.wikimedia.org

Wikimedia Foundation – l'article « mojibake » de Wikipédia en japonais, mal décodé (CC BY-SA 4.0)
commons.wikimedia.org

Katerina Chernoskutova – une carte postale envoyée en Russie (domaine public)
commons.wikimedia.org

Twitter et contributeurs (Twemoji) – les émojis (CC BY 4.0)
github.com